

Enci/ VENTAS

Manual de Despliegue

Publicacion Controlada - Backend API, Frontend Web, App Vendedor y Componentes Parciales

Version 1.0
Julio 2026
Aplicacion de Automatizacion de Fuerza de Ventas

Cliente	Enci / Enci Ventas
Representante receptor	Max
Equipo desarrollador	Grupo PLM - Universidad Tecnologica Metropolitana (UTEM)
Lider de equipo	Joshua Flores
Integrantes	Falon Berrios, Caroline Palacios y Eliseo Poblete
Repositorio oficial del MVP web	Chumajer14/Encipharm
Tipo de documento	Manual formal para despliegue, validacion, rollback y evidencia de salida

BACKEND API HTTPS	FRONTEND Vercel	APP Vendedor	UAT Go-live controlado
----------------------	--------------------	-----------------	---------------------------

Naturaleza del documento

Este manual define el procedimiento operativo para publicar el MVP web avanzado de Enci Ventas y validar su funcionamiento en ambientes de Preview, UAT o Produccion. No reemplaza credenciales, contratos de hosting ni aprobaciones internas del cliente; organiza el proceso tecnico, sus verificaciones y la evidencia minima que debe quedar registrada.

Tabla de Contenidos

1. Proposito y alcance del despliegue
2. Principios de salida controlada
3. Ambientes y responsabilidades
4. Preparacion de version antes del despliegue
5. Despliegue del backend FastAPI
6. Despliegue del frontend web en Vercel
7. Despliegue de app vendedor web/PWA
8. Entrega de app Flutter nativa parcial
9. Consideraciones para IA RAG
10. Smoke test posterior al despliegue
11. Checklist previo y posterior
12. Rollback y contingencia
13. Evidencia formal de entrega

1. Proposito y alcance del despliegue

El presente manual establece la secuencia tecnica para desplegar el sistema Enci Ventas en ambientes accesibles para validacion del cliente. El documento cubre el backend FastAPI, el frontend web React/Vite, la app vendedor web/PWA y las verificaciones necesarias para salida UAT o go-live controlado.

El procedimiento considera que el MVP web se entrega cerrado para Development y que la salida productiva depende de variables externas: dominios finales, credenciales del cliente, disponibilidad del proveedor cloud, usuarios reales, configuracion Firebase y aprobacion UAT.

Criterio de alcance

El despliegue no convierte componentes parciales en alcance completo. La IA RAG se entrega en su estado actual y la app Flutter nativa se trata como entregable parcial externo al repositorio, no como flujo de despliegue Vercel/GitHub.

1.1 Componentes cubiertos

Componente	Tecnologia	Procedimiento incluido	Resultado esperado
Backend API	FastAPI + Python + Firebase Admin	Publicacion en runtime HTTPS compatible, carga de variables, prueba de health/readiness	API disponible para frontend y pruebas protegidas
Frontend web	React + Vite	Deploy independiente en Vercel con Root Directory frontend	Pantalla login y navegacion web accesibles
App vendedor web/PWA	React/Vite en vendedor-app	Deploy separado en Vercel, opcionalmente con proxy /api	PWA de vendedor disponible para validacion
IA RAG	DeepSeek + Chroma + FastAPI	Configuracion backend, carga de secretos y prueba de /rag/chat	Chat documental parcial operativo si el ambiente tiene proveedor y corpus
Flutter nativo	Flutter + Firebase	Entrega de APK o build externo al repositorio	Aplicacion nativa Android parcialmente realizada, sin publicacion en tiendas

2. Principios de salida controlada

El despliegue debe ejecutarse como una actividad controlada. La publicacion de una URL no equivale automaticamente a aprobacion del sistema; solo habilita la validacion funcional, tecnica y de negocio definida para UAT.

- Congelar la rama, version o commit que sera publicado antes de iniciar la salida.
- Ejecutar validaciones locales minimas: build de frontend, lint, pruebas backend relevantes y revision de variables.
- No exponer secretos en repositorio, frontend, capturas o archivos compartidos.
- Usar HTTPS para cualquier API consumida por Vercel, Firebase Auth o app vendedor.
- Autorizar dominios finales en Firebase Authentication antes de validar login.
- Actualizar CORS backend con dominios reales de frontend, app vendedor y previews permitidos.
- Registrar fecha, responsable, commit/version, ambiente, URLs y resultado del smoke test.

2.1 Estados de despliegue

Estado	Uso	Permisos	Criterio para avanzar
Local	Pruebas del equipo antes de publicar	Solo equipo tecnico	Servicios levantan en 127.0.0.1 y pruebas base pasan

Estado	Uso	Permisos	Criterio para avanzar
Preview	Revision visual y funcional preliminar	Equipo y cliente segun URL compartida	Login, rutas y consumo API validan sin errores bloqueantes
UAT	Validacion formal con usuarios reales o cuentas autorizadas	Cliente, Max, equipo tecnico	Casos criticos ejecutados con evidencia
Production	Uso operativo o piloto controlado	Usuarios finales definidos	Aprobacion UAT, dominios finales y rollback definido

3. Ambientes y responsabilidades

La calidad del despliegue depende de separar responsabilidades. El equipo PLM prepara el artefacto tecnico, valida la configuracion y documenta la salida; el cliente debe proveer cuentas, dominios, aprobaciones y credenciales finales cuando corresponda.

Responsable	Responsabilidad principal	Evidencia
Grupo PLM	Preparar version, publicar componentes, ejecutar smoke test y registrar hallazgos	Commit, URLs, capturas, resultado de comandos
Joshua Flores	Coordinar salida, control de alcance, aprobacion tecnica interna y comunicacion con cliente	Registro de version y acta de despliegue
Max / Enci	Validar alcance, usuarios, reglas comerciales, dominios y aprobacion UAT	Observaciones, aceptacion o rechazo formal
Administrador de nube	Proveer credenciales, permisos de deploy y variables seguras	Variables cargadas, dominios, certificados

3.1 Prerrequisitos tecnicos

- Acceso al repositorio oficial del MVP web y a la rama/commit de salida.
- Cuenta de Vercel o plataforma equivalente con permisos para crear proyectos y variables.
- Proyecto Firebase con Google Authentication habilitado y dominios autorizados.
- Backend desplegable en Cloud Run, Render u otro runtime HTTPS compatible con FastAPI.
- Credencial Firebase Admin segura mediante service account o variable FIREBASE_SERVICE_ACCOUNT_JSON.
- Variables VITE_* disponibles para frontend y app vendedor, sin incluir secretos administrativos.
- Plan de rollback acordado antes de abrir el ambiente a validacion del cliente.

4. Preparacion de version antes del despliegue

Antes de publicar se debe fijar la version que sera evaluada. Esta practica evita que cambios posteriores alteren la evidencia UAT y permite revertir con precision ante un defecto bloqueante.

4.1 Congelamiento de salida

- 1 Seleccionar rama, tag o commit que representa la entrega.
- 2 Confirmar que no existan cambios locales sin commit en los componentes a desplegar.
- 3 Registrar hash de commit, fecha, responsable y ambiente objetivo.
- 4 Bloquear cambios no criticos hasta terminar smoke test y validacion UAT inicial.
- 5 Separar correcciones criticas en una rama de hotfix si aparecen defectos S1 o S2.

4.2 Validacion local minima

```
# Backend
cd Backend
uv sync
uv run pytest
uv run uvicorn app.main:app --host 127.0.0.1 --port 8000 --reload

# Frontend web
cd frontend
npm ci
npm run lint
npm run build

# App vendedor web/PWA
cd vendedor-app
npm ci
npm run lint
npm run build
```

No avanzar si
No debe ejecutarse un despliegue formal si el backend no inicia, el build frontend falla, faltan variables obligatorias, el dominio no esta autorizado en Firebase o el backend no puede responder /health y /readiness.

5. Despliegue del backend FastAPI

El backend es el componente central de autorizacion y negocio. Valida Firebase ID Tokens, consulta Firestore, aplica roles, controla CORS, expone endpoints comerciales y concentra los secretos de proveedores externos como DeepSeek.

5.1 Requisitos del runtime

Aspecto	Requisito
Protocolo	HTTPS publico para consumo desde Vercel, Firebase y usuarios externos
Python	Version compatible con el backend, preferentemente Python 3.12 o superior
Comando de arranque	uvicorn app.main:app con host 0.0.0.0 y puerto provisto por el runtime
Variables	APP_ENV, FIREBASE_PROJECT_ID, credencial Firebase Admin, CORS_ORIGINS y variables RAG si aplica
Seguridad	Swagger deshabilitado en produccion, CORS sin wildcard y role switcher temporal deshabilitado

5.2 Variables backend minimas

Variable	Obligatoria	Uso	Criterio
APP_NAME	Si	Nombre visible de API	Valor descriptivo del ambiente
APP_ENV	Si	Modo development, uat, staging o production	production debe bloquear /docs y openapi publico
APP_VERSION	Si	Version informativa	Debe coincidir con version de entrega
FIREBASE_PROJECT_ID	Si	Proyecto Firebase	No debe ser placeholder
GOOGLE_APPLICATION_CREDENTIALS	Condicional	Ruta a service account	Solo si el runtime usa archivo seguro
FIREBASE_SERVICE_ACCOUNT_JSON	Condicional	Service account como secreto	Preferible en hosting sin archivo persistente
CORS_ORIGINS	Si	Lista de dominios permitidos	Sin * en produccion
CORS_ORIGIN_REGEX	Opcional	Previews acotados de Vercel	Debe ser restrictivo
ENABLE_TEMPORARY_ROLE_SWITCHER	Si	Endpoint temporal de desarrollo	false fuera de development

5.3 Ejemplo de configuracion backend

```
APP_NAME=Enci Ventas API
APP_ENV=uat
APP_VERSION=1.0.0
FIREBASE_PROJECT_ID=enci-ventas-prod
FIREBASE_SERVICE_ACCOUNT_JSON={...secreto cargado en hosting...}
CORS_ORIGINS=https://encipharm.vercel.app,https://enciapp.vercel.app
CORS_ORIGIN_REGEX=^https://(encipharm|enciapp)-[a-z0-9-]+-chumajer14s-projects\.vercel\.app$
ENABLE_TEMPORARY_ROLE_SWITCHER=false
```

5.4 Comando de arranque recomendado

```
uv run uvicorn app.main:app --host 0.0.0.0 --port ${PORT:-8000}
```

En plataformas que no soporten la sintaxis anterior, se debe adaptar el puerto al mecanismo propio del proveedor. Lo relevante es que el servicio quede publicado por HTTPS y que el endpoint /health responda correctamente.

5.5 Verificaciones backend post deploy

Verificacion	Endpoint / accion	Resultado esperado
Servicio vivo	GET /health	Respuesta JSON con status=ok
Preparacion UAT	GET /readiness	status=ready o detalle explicito de checks pendientes
OpenAPI productivo	GET /docs	No expuesto si APP_ENV=production
Autenticacion	GET /me con Bearer token	Retorna uid, email, rol y activo
CORS	Request desde dominio Vercel	Respuesta permitida solo desde origen aprobado

6. Despliegue del frontend web en Vercel

El frontend web se publica como proyecto Vercel independiente, apuntando al directorio frontend. La aplicacion es React/Vite y consume el backend FastAPI mediante VITE_API_BASE_URL y tokens Bearer de Firebase.

6.1 Configuracion del proyecto

Campo Vercel	Valor
Root Directory	frontend
Framework Preset	Vite
Install Command	npm ci
Build Command	npm run build
Output Directory	dist
SPA Routing	Reescritura a index.html mediante vercel.json

6.2 Flujo recomendado

- 1 Crear proyecto en Vercel desde el repositorio oficial.
- 2 Configurar Root Directory como frontend.
- 3 Cargar variables VITE_* en Preview y Production.
- 4 Agregar el dominio generado por Vercel en Firebase Authentication.

- 5 Agregar el mismo dominio en CORS_ORIGINS del backend.
- 6 Ejecutar deploy Preview y validar login, rutas y consumo de API.
- 7 Ejecutar deploy Production solo despues de aprobar Preview o UAT inicial.

6.3 Variables frontend

```
VITE_APP_NAME=Enci Ventas
VITE_API_BASE_URL=https://api-publica.enci.example
VITE_FIREBASE_API_KEY=...
VITE_FIREBASE_AUTH_DOMAIN=...
VITE_FIREBASE_PROJECT_ID=...
VITE_FIREBASE_STORAGE_BUCKET=...
VITE_FIREBASE_MESSAGING_SENDER_ID=...
VITE_FIREBASE_APP_ID=...
VITE_FIREBASE_MEASUREMENT_ID=...
VITE_FIREBASE_FREE_TIER_MODE=true
```

Advertencia frontend

VITE_API_BASE_URL no puede apuntar a localhost cuando la app esta publicada en Vercel. Debe apuntar a una API HTTPS accesible desde internet o al proxy autorizado definido para el proyecto.

7. Despliegue de app vendedor web/PWA

La app vendedor web/PWA puede publicarse como proyecto Vercel separado desde el directorio vendedor-app. Esta aplicacion no sustituye la app Flutter nativa parcial; corresponde al componente web/PWA versionado en el repositorio.

Campo	Valor recomendado
Proyecto Vercel	enciapp o nombre equivalente aprobado
Root Directory	vendedor-app
Framework Preset	Vite
Install Command	npm ci
Build Command	npm run build
Output Directory	dist
API	VITE_API_BASE_URL=https://api-publica.example.com o proxy /api

7.1 Validacion funcional minima

- Abrir la URL final desde navegador desktop y movil.
- Iniciar sesion con Google usando usuario autorizado en Firebase.
- Confirmar que el backend reconoce el token y responde endpoints protegidos.
- Crear o simular una cotizacion segun datos disponibles.
- Verificar acceso a pipeline, forecast y Enci Chat cuando el ambiente lo permita.
- Instalar la PWA desde Chrome/Android y validar modo standalone.

8. Entrega de app Flutter nativa parcial

La aplicacion Flutter nativa Android se considera un entregable parcial externo al monorepo. Su ausencia en el repositorio oficial no invalida su condicion de componente desarrollado fuera de GitHub, conforme al alcance conversado con Max y a su aceptacion parcial.

Aspecto	Criterio de entrega
Formato	APK instalable o build Android equivalente, segun disponibilidad del equipo
Estado	App nativa parcialmente realizada, con UI y login Firebase funcional segun alcance informado
Backend	No se garantiza cobertura total de endpoints FastAPI desde Flutter
Publicacion	No incluye Google Play, App Store, certificados comerciales, MDM ni mantenimiento post entrega
Validacion	Instalacion en dispositivo Android, apertura, login y navegacion principal
Cierre	Solicitudes futuras se consideran nuevo requerimiento, mejora o Fase 2

Limite formal

El procedimiento de despliegue cloud no aplica a Flutter si el entregable se entrega como APK. Para publicacion en tiendas, firma definitiva, certificados, politicas de privacidad o mantenimiento se requiere nuevo alcance.

9. Consideraciones para IA RAG

La IA RAG se despliega como parte del backend. Su operacion depende de variables privadas, proveedor externo, almacenamiento documental y disponibilidad de indice local o persistente.

Variable / recurso	Uso	Ambiente
DEEPSEEK_API_KEY	Llave del proveedor LLM	Solo backend
DEEPSEEK_BASE_URL	Base URL del proveedor	Backend
DEEPSEEK_MODEL	Modelo usado por el backend	Backend
CHROMA_PERSIST_DIR	Persistencia del indice vectorial	Backend
GCS_BUCKET_DOCUMENTS	Bucket de documentos en produccion si aplica	Backend / GCP
LOCAL_DOCUMENT_STORAGE_DIR	Directorio local para desarrollo	Solo local o UAT controlado
RAG_CHAT_RATE_LIMIT_PER_MINUTE	Limite por usuario	Backend
RAG_MAX_UPLOAD_BYTES	Tamaño maximo de carga documental	Backend

9.1 Prueba minima RAG

- 1 Confirmar que DEEPSEEK_API_KEY exista solo en backend.
- 2 Iniciar sesion con usuario autorizado.
- 3 Ejecutar POST /rag/chat desde frontend, app vendedor o Swagger controlado.
- 4 Revisar que la respuesta incluya conversacion_id, respuesta, fuentes cuando exista contexto y timestamp.
- 5 Si no existe corpus o contexto, aceptar respuesta local sin invocar proveedor externo.

Alcance RAG

El despliegue no incluye entrenamiento adicional, tuning de prompts, cambio de proveedor, evaluacion avanzada de calidad, subida masiva de corpus ni SLA del proveedor externo. Se entrega el estado funcional disponible dentro del alcance parcial pactado.

10. Smoke test posterior al despliegue

Despues de publicar cualquier ambiente, se debe ejecutar un smoke test que confirme que el sistema esta vivo, que el frontend carga, que la autenticacion funciona y que los flujos comerciales criticos no presentan errores bloqueantes.

10.1 Comandos de referencia

```
# Validacion API sin token
curl https://api.example.com/health
curl https://api.example.com/readiness

# Smoke test automatizado si la herramienta esta disponible
python tools/smoke_crm_web.py --env uat --api-base-url https://api.example.com --web-url https://crm.example.com

# Smoke test con token Firebase
python tools/smoke_crm_web.py --env uat --api-base-url https://api.example.com --web-url https://crm.example.com --token <firebase_id_token>
```

10.2 Matriz de smoke test

Paso	Accion	Resultado esperado	Bloqueante
ST-01	Abrir frontend web	Pantalla de login visible	Si
ST-02	Login Google	Usuario autenticado y redirigido	Si
ST-03	GET /health	status=ok	Si
ST-04	GET /readiness	status=ready o checks explicitados	Si para go-live
ST-05	GET /me	uid, email, rol y activo	Si
ST-06	Clientes	Alta, listado o lectura segun rol	Si
ST-07	Oportunidades	Creacion/listado funcional	Si
ST-08	RAG	Respuesta local o DeepSeek controlada	No si esta declarado parcial
ST-09	App vendedor	Login, navegacion y flujo base	Segun alcance

11. Checklist previo y posterior

11.1 Checklist previo

Item	Estado requerido
Rama o commit congelado	Registrado antes de desplegar
Variables backend	Cargadas sin placeholders ni secretos expuestos
Variables frontend	Cargadas en Preview y Production si aplica
Firebase Auth	Dominios Vercel autorizados
CORS backend	Dominios finales y previews permitidos
Build frontend	npm run build exitoso
Backend tests	Suite relevante en verde o excepcion documentada
Swagger production	No expuesto publicamente
Plan rollback	Definido antes de go-live

11.2 Checklist posterior

Item	Evidencia esperada
URL frontend	Carga login y rutas principales

Item	Evidencia esperada
URL backend	/health y /readiness responden
Login real	Token Firebase aceptado por backend
Usuarios	Rol y estado activo aplicados
Flujo comercial	Cliente, oportunidad y propuesta segun rol
App vendedor	URL PWA o APK validado segun alcance
RAG	Respuesta controlada o limitacion documentada
Observaciones	Registradas con severidad y decision

12. Rollback y contingencia

Rollback debe ejecutarse cuando aparezca una falla S1 o S2 sin mitigacion inmediata. El objetivo no es corregir en caliente sin control, sino volver a un estado estable, registrar el incidente y reintentar con una version corregida.

12.1 Causales de rollback

- Login bloqueado para usuarios validos.
- Backend caido o sin respuesta en /health.
- Errores de autorizacion que exponen datos de otro vendedor.
- Corrupcion, perdida o escritura incorrecta de datos comerciales.
- CORS mal configurado que impide toda comunicacion frontend-backend.
- Deploy que expone secretos o habilita funcionalidades temporales de desarrollo en produccion.

12.2 Procedimiento de rollback

- 1 Detener nuevas pruebas o uso operativo del ambiente afectado.
- 2 Identificar version estable previa del backend y del frontend.
- 3 Revertir deploy en plataforma correspondiente: Vercel, Cloud Run, Render u otra.
- 4 Confirmar que /health y /readiness vuelven a responder.
- 5 Validar login con usuario autorizado.
- 6 Registrar hora, version revertida, version restaurada, responsable y causa.
- 7 Crear hotfix o tarea correctiva antes de nuevo intento de salida.

13. Evidencia formal de entrega

Cada despliegue entregado al cliente debe dejar evidencia tecnica y operativa suficiente para trazabilidad. La evidencia permite confirmar que el ambiente publicado corresponde a la version evaluada y que las observaciones posteriores se analizan contra un punto fijo.

Evidencia	Descripcion	Responsable
URL frontend web	Dominio Vercel o dominio final del CRM	Grupo PLM
URL backend	Base URL HTTPS de API	Grupo PLM / administrador cloud
URL app vendedor	Vercel PWA o evidencia APK Flutter	Grupo PLM
Commit/version	Hash, rama, tag o version publicada	Grupo PLM

Evidencia	Descripcion	Responsable
Variables revisadas	Confirmacion sin exponer valores secretos	Responsable tecnico
Smoke test	Fecha, resultado y observaciones	Grupo PLM
UAT	Observaciones de Max/Enci y decision	Cliente
Rollback	Procedimiento disponible y version estable previa	Grupo PLM

Condicion de cierre

Un despliegue se considera listo para revision del cliente cuando las URLs estan disponibles, el smoke test base esta ejecutado, las variables no contienen placeholders criticos, Firebase y CORS aceptan los dominios publicados, y toda limitacion parcial queda documentada como RAG, Flutter, UAT, infraestructura o Fase 2.